# La firma de Schnorr con números pequeños

Dos ejemplos de juguete para reproducir a mano:
en $\mathbb{Z}_{23}^*$ y en una curva elíptica sobre $\mathbb{F}_{17}$

José de Jesús Angel Angel

Lab-IA · Facultad de Ingeniería, Universidad Anáhuac México

13 de octubre de 2026

**Resumen**

Este documento ofrece una primera visión de la firma de Schnorr mediante dos ejemplos completos con números tan pequeños que todas las cuentas se pueden hacer con lápiz y papel. El primero trabaja con potencias módulo $p = 23$; el segundo, con puntos de la curva $y^2 = x^3 + 2x + 2$ sobre $\mathbb{F}_{17}$. Ambos siguen exactamente los mismos pasos (generar claves, firmar, verificar) y muestran, además, qué ocurre si se altera el mensaje y por qué nunca debe reutilizarse el número aleatorio $k$. Después se construye, en ambos grupos, una multifirma de dos participantes (MuSig) y se cierra con un resumen de cómo se usa Schnorr en la práctica: umbrales, Taproot, firmas adaptadoras y contratos inteligentes. Una sección final presenta las firmas digitales especiales (ciegas, de anillo, de grupo, agregadas, de credenciales, poscuánticas y otras) y el problema que resuelve cada una. Se incluyen tablas auxiliares, código en Python para comprobar los resultados, ejercicios con solución y una tarea con su programa verificador.

## Índice general

**1 El esquema en una página** **3**
1.1 La idea sin fórmulas . . . 3
1.2 El algoritmo . . . 3
1.3 El hash de juguete . . . 3

**2 Ejemplo 1: Schnorr con aritmética exponencial en $\mathbb{Z}_{23}^*$** **4**
2.1 Parámetros . . . 4
2.2 Generación de claves . . . 4
2.3 Firma . . . 4
2.4 Verificación . . . 5
2.5 ¿Qué pasa si alguien altera el mensaje? . . . 5
2.6 ¿Por qué nunca se reutiliza $k$? . . . 5
2.7 Comprobación en Python . . . 6

**3 De potencias a puntos: lo mínimo de curvas elípticas** **6**
3.1 La curva y su grupo . . . 6
3.2 Diccionario entre los dos mundos . . . 7

**4 Ejemplo 2: Schnorr en la curva $y^2 = x^3 + 2x + 2$ sobre $\mathbb{F}_{17}$ 7**
4.1 Parámetros . . . 7
4.2 Cómo se calculan los múltiplos de $G$ . . . 7
4.3 Hash de juguete para puntos . . . 9
4.4 Generación de claves . . . 9
4.5 Firma . . . 9
4.6 Verificación . . . 9
4.7 Mensaje alterado y reutilización de $k$ . . . 10
4.8 Comprobación en Python . . . 11

**5 Los dos ejemplos lado a lado 11**
5.1 Del juguete a la realidad . . . 12

**6 Multifirma 12**
6.1 La idea . . . 12
6.2 Ejemplo en $\mathbb{Z}_{23}^*$ con dos firmantes . . . 13
6.3 Ejemplo en la curva $y^2 = x^3 + 2x + 2$ sobre $\mathbb{F}_{17}$ con dos firmantes . . . 15
6.4 Comprobación en Python . . . 16

**7 Ejercicios 17**

**8 Tarea 18**

**9 Schnorr en la realidad: resumen, variantes y aplicaciones 21**
9.1 La firma individual en producción . . . 21
9.2 ¿Solo existe la multifirma "$n$ de $n$"? . . . 22
9.3 Otras construcciones con aplicación real . . . 23
9.4 Schnorr en criptomonedas y contratos inteligentes . . . 23
9.5 Limitaciones y frontera . . . 24

**10 Firmas digitales especiales 25**
10.1 Privacidad del firmante . . . 25
10.2 Control sobre quién puede verificar . . . 26
10.3 Delegación y firma colectiva . . . 26
10.4 Credenciales y divulgación selectiva . . . 27
10.5 Gestión de claves y seguridad a largo plazo . . . 28
10.6 Resumen . . . 29

# 1 El esquema en una página

## 1.1 La idea sin fórmulas

La firma de Schnorr es una forma de demostrar que uno conoce un secreto $x$ sin revelarlo, atada a un mensaje concreto. Tiene tres movimientos:

1. **Compromiso.** El firmante elige un número al azar $k$ y publica $r = g^k$. Es como mostrar un sobre cerrado.
2. **Reto.** El reto $e$ se obtiene aplicando una función hash a $r$ y al mensaje. Nadie lo elige: lo "elige" el hash.
3. **Respuesta.** El firmante responde $s = k + x\,e$. Sin conocer $x$, es prácticamente imposible dar una respuesta que pase la verificación; y como $k$ es aleatorio, $s$ no revela $x$.

## 1.2 El algoritmo

**Definición: firma de Schnorr**

**Parámetros.** Un grupo $\mathbb{G}$ de orden primo $q$ con generador $g$, en el que el logaritmo discreto es difícil, y una función hash $H : \{0,1\}^* \to \mathbb{Z}_q$.
**Generación de claves.** Elegir la clave privada $x \in \mathbb{Z}_q$. La clave pública es $y = g^{-x}$.
**Firma** de un mensaje $M$:

1. Elegir $k \in \mathbb{Z}_q$ al azar.
2. Calcular $r = g^k$.
3. Calcular $e = H(r \,\|\, M)$.
4. Calcular $s = k + x\,e \bmod q$.

La firma es el par $(s, e)$.
**Verificación.** Calcular $r_v = g^s\, y^e$ y $e_v = H(r_v \,\|\, M)$. La firma es válida si y solo si $e_v = e$.

**Por qué funciona.** Si la firma se hizo correctamente,

$$r_v = g^s\, y^e = g^{\,k+xe} \cdot g^{-xe} = g^k = r,$$

y por lo tanto $e_v = H(r \,\|\, M) = e$.

**Por qué los exponentes se reducen módulo $q$.** Como $g$ tiene orden $q$, se cumple $g^q = 1$, de modo que $g^a = g^{a \bmod q}$. Por eso $s$, $e$, $x$ y $k$ viven en $\mathbb{Z}_q$, aunque los elementos del grupo ($g$, $y$, $r$) vivan en otro conjunto ($\mathbb{Z}_p^*$ en el primer ejemplo, puntos de una curva en el segundo).

## 1.3 El hash de juguete

Una función hash real, como SHA-256, no se puede calcular a mano. Para estos ejemplos usaremos una versión de juguete que imita la concatenación $\|$:

**Definición: hash de juguete**

$H(a \,\|\, b \,\|\, \cdots)$ es el número que se obtiene al *escribir seguidos* los dígitos decimales de $a, b, \ldots$, reducido módulo $q$.

Por ejemplo, con $q = 11$: $H(13 \,\|\, 14) = 1314 \bmod 11 = 5$, pues $1314 = 11 \cdot 119 + 5$.

**Advertencia: solo didáctico**

Los grupos de estos ejemplos tienen 11 y 19 elementos: el logaritmo discreto se resuelve probando todos los valores, y el hash de juguete no tiene ninguna propiedad criptográfica. Los ejemplos sirven para entender la mecánica del esquema, no ofrecen seguridad alguna.

# 2 Ejemplo 1: Schnorr con aritmética exponencial en $\mathbb{Z}_{23}^*$

## 2.1 Parámetros

- Primo $p = 23$. Trabajamos con los números $\{1, \ldots, 22\}$ y la multiplicación módulo 23.
- Orden $q = 11$. Como $p - 1 = 22 = 2 \cdot 11$, existe un subgrupo de orden primo 11.
- Generador $g = 2$. Su orden es 11: $2^{11} = 2048 = 89 \cdot 23 + 1$, así que $2^{11} \equiv 1 \pmod{23}$.

La Tabla 1 contiene las once potencias de $g$. Cada entrada se obtiene de la anterior multiplicando por 2 y reduciendo módulo 23. Con ella, y con la tabla de inversos módulo 11, todas las cuentas del ejemplo se hacen en pocos minutos.

| $a$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $2^a \bmod 23$ | 1 | 2 | 4 | 8 | 16 | 9 | 18 | 13 | 3 | 6 | 12 |

| $a$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| $a^{-1} \bmod 11$ | 1 | 6 | 4 | 3 | 9 | 2 | 8 | 7 | 5 | 10 |

Tabla 1: Potencias de $g = 2$ módulo 23 e inversos módulo 11.

## 2.2 Generación de claves

**Paso 1: claves**

- Clave privada: $x = 6$.
- Clave pública: $y = g^{-x} = g^{-6}$. Como los exponentes se reducen módulo 11, $-6 \equiv 5$, así que
$$y = g^5 = 2^5 = 32 \equiv 9 \pmod{23}.$$
- Comprobación: $g^6 \cdot y = 18 \cdot 9 = 162 = 7 \cdot 23 + 1 \equiv 1$. En efecto, $y$ es el inverso de $g^x$.

## 2.3 Firma

El mensaje es $M = 14$ (por ejemplo, "pagar 14 monedas").

**Paso 2: firmar $M = 14$**

1. Número aleatorio: $k = 7$.
2. Compromiso: $r = g^7 = 2^7 = 128 = 5 \cdot 23 + 13$, es decir, $r = 13$.

3. Reto: $e = H(13 \,\|\, 14) = 1314 \bmod 11$. Como $1314 = 11 \cdot 119 + 5$, resulta $e = 5$.
4. Respuesta: $s = k + x\,e = 7 + 6 \cdot 5 = 37 = 3 \cdot 11 + 4$, es decir, $s = 4$.

**Firma de $M = 14$:** $(s, e) = (4,\, 5)$

## 2.4 Verificación

El verificador conoce $g = 2$, $p = 23$, $q = 11$, la clave pública $y = 9$, el mensaje $M = 14$ y la firma $(4, 5)$. *No* conoce $x$ ni $k$.

**Paso 3: verificar**

1. $g^s = 2^4 = 16$.
2. $y^e = 9^5$, por elevación al cuadrado repetida:

$$\begin{aligned} 9^2 &= 81 = 3 \cdot 23 + 12 \equiv 12, \\ 9^4 &= 12^2 = 144 = 6 \cdot 23 + 6 \equiv 6, \\ 9^5 &= 9^4 \cdot 9 = 54 = 2 \cdot 23 + 8 \equiv 8. \end{aligned}$$

3. $r_v = g^s\, y^e = 16 \cdot 8 = 128 \equiv 13 \pmod{23}$.
4. $e_v = H(13 \,\|\, 14) = 1314 \bmod 11 = 5$.

Como $e_v = 5 = e$, la firma es **válida**. Observe que $r_v = 13$ coincide con el $r$ del firmante, aunque el verificador nunca lo vio.

## 2.5 ¿Qué pasa si alguien altera el mensaje?

Supongamos que un atacante cambia el mensaje a $M' = 15$ y conserva la firma $(4, 5)$. El verificador obtiene el mismo $r_v = 13$, porque este solo depende de $s$, $e$ e $y$, pero ahora

$$e_v = H(13 \,\|\, 15) = 1315 \bmod 11 = 6 \neq 5.$$

La firma se **rechaza**. Para falsificarla, el atacante tendría que encontrar un $s'$ con $H(g^{s'}\, y^e \,\|\, 15) = e$, lo cual exige, en tamaños reales, conocer $x$.

## 2.6 ¿Por qué nunca se reutiliza $k$?

Supongamos que el firmante, por descuido, firma otro mensaje $M_2 = 16$ con el mismo $k = 7$:

$$r = 13, \qquad e_2 = H(13 \,\|\, 16) = 1316 \bmod 11 = 7, \qquad s_2 = 7 + 6 \cdot 7 = 49 \equiv 5 \pmod{11}.$$

Un observador ve dos firmas, $(s_1, e_1) = (4, 5)$ y $(s_2, e_2) = (5, 7)$. Restando las ecuaciones $s_1 = k + x e_1$ y $s_2 = k + x e_2$, el $k$ desaparece:

$$x = \frac{s_1 - s_2}{e_1 - e_2} = \frac{4 - 5}{5 - 7} = \frac{-1}{-2} \equiv \frac{10}{9} = 10 \cdot 9^{-1} = 10 \cdot 5 = 50 \equiv 6 \pmod{11}.$$

**Idea clave**

Con dos firmas que comparten $k$, cualquiera recupera la clave privada con una resta y una división. El mismo error, en su versión para ECDSA, permitió extraer la clave de firma de la PlayStation 3 (2010) y vaciar billeteras de Bitcoin en Android (2013), cuyo generador aleatorio repetía nonces.

### 2.7 Comprobación en Python

```
p, q, g = 23, 11, 2
H = lambda *partes: int("".join(str(a) for a in partes)) % q

x = 6
y = pow(g, -x, p)                # 9
M, k = 14, 7
r = pow(g, k, p)                 # 13
e = H(r, M)                      # 5
s = (k + x * e) % q              # 4

rv = pow(g, s, p) * pow(y, e, p) % p   # 13
assert H(rv, M) == e                    # firma válida
assert H(rv, 15) != e                   # mensaje alterado: rechazo
```

## 3 De potencias a puntos: lo mínimo de curvas elípticas

### 3.1 La curva y su grupo

Una curva elíptica sobre $\mathbb{F}_p$ es el conjunto de pares $(x, y)$ con $x, y \in \mathbb{F}_p$ que satisfacen

$$E:\ y^2 = x^3 + a\,x + b \pmod{p},$$

junto con un punto especial $\mathcal{O}$, el *punto al infinito*, que hace el papel del neutro (el "1" del grupo). Los puntos se *suman* con las siguientes reglas.

**Definición: ley de grupo**

Sean $P = (x_1, y_1)$ y $Q = (x_2, y_2)$.

- **Neutro:** $P + \mathcal{O} = \mathcal{O} + P = P$.
- **Opuesto:** $-P = (x_1, -y_1)$, y $P + (-P) = \mathcal{O}$. Esto incluye el caso $x_1 = x_2$, $y_1 = -y_2$, y el doblado de un punto con $y_1 = 0$.
- **Suma** ($x_1 \neq x_2$): pendiente $\lambda = \dfrac{y_2 - y_1}{x_2 - x_1}$.
- **Doblado** ($P = Q$, $y_1 \neq 0$): pendiente $\lambda = \dfrac{3x_1^2 + a}{2y_1}$.

En los dos últimos casos, $P + Q = (x_3, y_3)$ con

$$x_3 = \lambda^2 - x_1 - x_2, \qquad y_3 = \lambda\,(x_1 - x_3) - y_1.$$

Todas las operaciones se hacen módulo $p$; dividir significa multiplicar por el inverso módulo $p$.

### 3.2 Diccionario entre los dos mundos

El esquema de Schnorr es idéntico; solo cambia la forma de escribir la operación del grupo.

| **Concepto** | **En $\mathbb{Z}_p^*$ (multiplicativo)** | **En la curva (aditivo)** |
|---|---|---|
| Operación del grupo | $a \cdot b \bmod p$ | $P + Q$ |
| Neutro | $1$ | $\mathcal{O}$ |
| Generador | $g$ | $G$ |
| Repetir la operación $k$ veces | $g^k$ | $kG$ |
| Inverso | $g^{-1}$ | $-G$ |
| Clave pública | $y = g^{-x}$ | $Y = -xG$ |
| Compromiso | $r = g^k$ | $R = kG$ |
| Verificación | $r_v = g^s\, y^e$ | $R_v = sG + eY$ |
| Método eficiente | elevación al cuadrado repetida | doblado y suma |

## 4 Ejemplo 2: Schnorr en la curva $y^2 = x^3 + 2x + 2$ sobre $\mathbb{F}_{17}$

### 4.1 Parámetros

- Campo $\mathbb{F}_{17}$: los números $0, \ldots, 16$ con aritmética módulo 17.
- Curva $E:\ y^2 = x^3 + 2x + 2$ [2].
- La curva tiene 18 puntos afines más $\mathcal{O}$, es decir, 19 puntos. Como 19 es primo, *cualquier* punto distinto de $\mathcal{O}$ genera todo el grupo. Tomamos $q = 19$.
- Generador $G = (5, 1)$. Está en la curva: $1^2 = 1$ y $5^3 + 2 \cdot 5 + 2 = 137 = 8 \cdot 17 + 1 \equiv 1$.

### 4.2 Cómo se calculan los múltiplos de $G$

Necesitaremos inversos módulo 17 (Tabla 2). Veamos dos cálculos completos, uno de cada tipo.

| $a$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $a^{-1} \bmod 17$ | 1 | 9 | 6 | 13 | 7 | 3 | 5 | 15 | 2 | 12 | 14 | 10 | 4 | 11 | 8 | 16 |

| $a$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $a^{-1} \bmod 19$ | 1 | 10 | 13 | 5 | 4 | 16 | 11 | 12 | 17 | 2 | 7 | 8 | 3 | 15 | 14 | 6 | 9 | 18 |

Tabla 2: Inversos módulo 17 (para las coordenadas) y módulo 19 (para los escalares).

**Doblado: $2G$ con $G = (5, 1)$**

$$\lambda = \frac{3 \cdot 5^2 + 2}{2 \cdot 1} = \frac{77}{2} \equiv \frac{9}{2} = 9 \cdot 2^{-1} = 9 \cdot 9 = 81 \equiv 13 \pmod{17}.$$

$$x_3 = 13^2 - 5 - 5 = 159 \equiv 6, \qquad y_3 = 13\,(5 - 6) - 1 = -14 \equiv 3.$$

Resultado: $2G = (6, 3)$. Comprobación: $3^2 = 9$ y $6^3 + 12 + 2 = 230 = 13 \cdot 17 + 9$.

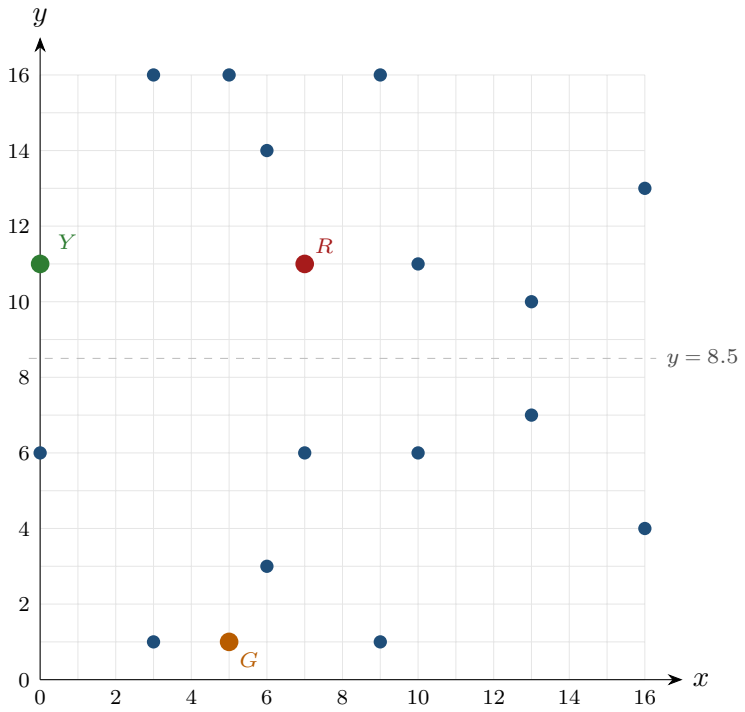



Figura 1: Los 18 puntos afines de $E(\mathbb{F}_{17})$. Son simétricos respecto a la recta $y = 8.5$, porque $(x, y)$ y $(x, 17 - y) = (x, -y)$ son opuestos. Se destacan el generador $G$, la clave pública $Y$ y el compromiso $R$ del ejemplo.

**Suma:** $3G = 2G + G = (6, 3) + (5, 1)$

$$\lambda = \frac{1-3}{5-6} = \frac{-2}{-1} = 2, \qquad x_3 = 2^2 - 6 - 5 = -7 \equiv 10, \qquad y_3 = 2\,(6 - 10) - 3 = -11 \equiv 6.$$

Resultado: $3G = (10, 6)$.

Repitiendo la suma con $G$ se obtiene la Tabla 3. Es una buena práctica completarla a mano y compararla.

| $k$ | $kG$ | $k$ | $kG$ | $k$ | $kG$ | $k$ | $kG$ |
|---|---|---|---|---|---|---|---|
| 1 | $(5, 1)$ | 6 | $(16, 13)$ | 11 | $(13, 10)$ | 16 | $(10, 11)$ |
| 2 | $(6, 3)$ | 7 | $(0, 6)$ | 12 | $(0, 11)$ | 17 | $(6, 14)$ |
| 3 | $(10, 6)$ | 8 | $(13, 7)$ | 13 | $(16, 4)$ | 18 | $(5, 16)$ |
| 4 | $(3, 1)$ | 9 | $(7, 6)$ | 14 | $(9, 1)$ | 19 | $\mathcal{O}$ |
| 5 | $(9, 16)$ | 10 | $(7, 11)$ | 15 | $(3, 16)$ | | |

Tabla 3: Múltiplos de $G = (5, 1)$. Note que $(19 - k)G = -kG$: por ejemplo, $18G = (5, 16) = -G$.

## 4.3 Hash de juguete para puntos

El compromiso ahora es un punto $R = (x_R, y_R)$. Lo concatenamos completo con el mensaje:

$$H(R \,\|\, M) = (\text{número formado por } x_R,\ y_R,\ M) \bmod 19.$$

## 4.4 Generación de claves

**Paso 1: claves**

- Clave privada: $x = 7$.
- Clave pública: $Y = -7G$. De la tabla, $7G = (0, 6)$, y su opuesto es $Y = (0, -6) = (0, 11)$. (Equivalentemente, $Y = 12G$, pues $-7 \equiv 12 \pmod{19}$.)
- Comprobación: $11^2 = 121 = 7 \cdot 17 + 2 \equiv 2$ y $0^3 + 2 \cdot 0 + 2 = 2$. El punto está en la curva.

## 4.5 Firma

**Paso 2: firmar $M = 14$**

1. Número aleatorio: $k = 10$.
2. Compromiso: $R = 10G = (7, 11)$, de la tabla.
3. Reto: $e = H(7 \,\|\, 11 \,\|\, 14) = 71114 \bmod 19$. Como $71114 = 19 \cdot 3742 + 16$, resulta $e = 16$.
4. Respuesta: $s = k + x\,e = 10 + 7 \cdot 16 = 122 = 6 \cdot 19 + 8$, es decir, $s = 8$.

**Firma de $M = 14$:** $(s, e) = (8,\ 16)$

## 4.6 Verificación

El verificador conoce la curva, $G = (5, 1)$, $q = 19$, la clave pública $Y = (0, 11)$, el mensaje $M = 14$ y la firma $(8, 16)$. *No* conoce $x$ ni $k$, así que no puede usar la tabla para $Y$: debe calcular $16Y$ a partir de $Y$. Por suerte, $s = 8 = 2^3$ y $e = 16 = 2^4$, de modo que basta con doblar.

**Paso 3a: $sG = 8G$ con tres doblados**

$2G = (6, 3)$ se calculó arriba. Después:

$$4G = 2 \cdot (6,3): \quad \lambda = \frac{3 \cdot 36 + 2}{2 \cdot 3} = \frac{110}{6} \equiv \frac{8}{6} = 8 \cdot 3 = 24 \equiv 7,$$
$$x_3 = 49 - 12 = 37 \equiv 3, \qquad y_3 = 7\,(6 - 3) - 3 = 18 \equiv 1;$$
$$8G = 2 \cdot (3,1): \quad \lambda = \frac{3 \cdot 9 + 2}{2 \cdot 1} = \frac{29}{2} \equiv \frac{12}{2} = 12 \cdot 9 = 108 \equiv 6,$$
$$x_3 = 36 - 6 = 30 \equiv 13, \qquad y_3 = 6\,(3 - 13) - 1 = -61 \equiv 7.$$

Resultado: $4G = (3, 1)$ y $\boxed{8G = (13, 7)}$.

**Paso 3b: $eY = 16Y$ con cuatro doblados, $Y = (0, 11)$**

$$2Y: \quad \lambda = \frac{3\cdot 0 + 2}{2\cdot 11} = \frac{2}{22} \equiv \frac{2}{5} = 2\cdot 7 = 14,$$
$$x_3 = 196 - 0 - 0 \equiv 9, \qquad y_3 = 14\,(0 - 9) - 11 = -137 \equiv 16;$$

$$4Y: \quad \lambda = \frac{3\cdot 81 + 2}{2\cdot 16} = \frac{245}{32} \equiv \frac{7}{15} = 7\cdot 8 = 56 \equiv 5,$$
$$x_3 = 25 - 18 = 7, \qquad y_3 = 5\,(9 - 7) - 16 = -6 \equiv 11;$$

$$8Y: \quad \lambda = \frac{3\cdot 49 + 2}{2\cdot 11} = \frac{149}{22} \equiv \frac{13}{5} = 13\cdot 7 = 91 \equiv 6,$$
$$x_3 = 36 - 14 = 22 \equiv 5, \qquad y_3 = 6\,(7 - 5) - 11 = 1.$$

Así, $2Y = (9, 16)$, $4Y = (7, 11)$, $8Y = (5, 1)$ y, como el doblado de $(5, 1)$ ya se hizo al calcular $2G$, $\boxed{16Y = (6, 3)}$.

**Paso 3c: $R_v = sG + eY$ y comparación del reto**

$$R_v = (13, 7) + (6, 3): \quad \lambda = \frac{3 - 7}{6 - 13} = \frac{-4}{-7} \equiv \frac{13}{10} = 13\cdot 12 = 156 \equiv 3.$$
$$x_3 = 9 - 13 - 6 = -10 \equiv 7, \qquad y_3 = 3\,(13 - 7) - 7 = 11.$$

Así, $R_v = (7, 11)$ y

$$e_v = H(7 \,\|\, 11 \,\|\, 14) = 71114 \bmod 19 = 16 = e.$$

La firma es **válida**.

**Comprobación con la tabla.** Quien conoce que $Y = 12G$ puede verificar el resultado con aritmética de escalares: $8G + 16\cdot 12G = (8 + 192)G = 200G$, y $200 = 10\cdot 19 + 10$, así que $R_v = 10G = R$. Esta comprobación usa el logaritmo discreto de $Y$, que en este juguete se lee en la tabla; en tamaños reales es justamente lo que nadie puede calcular.

## 4.7 Mensaje alterado y reutilización de $k$

**Mensaje alterado.** Con $M' = 15$ y la misma firma, el verificador obtiene el mismo $R_v = (7, 11)$, pero $e_v = H(7 \,\|\, 11 \,\|\, 15) = 71115 \bmod 19 = 17 \neq 16$. La firma se rechaza.

**Reutilización de $k$.** Si el firmante firma $M_2 = 16$ con el mismo $k = 10$:

$$e_2 = H(7 \,\|\, 11 \,\|\, 16) = 71116 \bmod 19 = 18, \qquad s_2 = 10 + 7\cdot 18 = 136 \equiv 3 \pmod{19}.$$

Con las firmas $(8, 16)$ y $(3, 18)$:

$$x = \frac{s_1 - s_2}{e_1 - e_2} = \frac{8 - 3}{16 - 18} = \frac{5}{-2} \equiv \frac{5}{17} = 5\cdot 9 = 45 \equiv 7 \pmod{19}.$$

La clave privada queda expuesta, exactamente como en el primer ejemplo.

### 4.8 Comprobación en Python

```
P, A, q = 17, 2, 19              # campo, coeficiente a, orden
G = (5, 1)

def suma(Pt, Qt):
    if Pt is None: return Qt
    if Qt is None: return Pt
    (x1, y1), (x2, y2) = Pt, Qt
    if x1 == x2 and (y1 + y2) % P == 0: return None   # punto al infinito
    if Pt == Qt: lam = (3*x1*x1 + A) * pow(2*y1, -1, P) % P
    else:        lam = (y2 - y1) * pow(x2 - x1, -1, P) % P
    x3 = (lam*lam - x1 - x2) % P
    return (x3, (lam*(x1 - x3) - y1) % P)

def mult(n, Pt):                  # doblado y suma
    R = None
    for b in bin(n)[2:]:
        R = suma(R, R)
        if b == "1": R = suma(R, Pt)
    return R

H = lambda *partes: int("".join(str(a) for a in partes)) % q
neg = lambda Pt: (Pt[0], -Pt[1] % P)

x = 7
Y = neg(mult(x, G))              # (0, 11)
M, k = 14, 10
R = mult(k, G)                   # (7, 11)
e = H(R[0], R[1], M)             # 16
s = (k + x * e) % q              # 8

Rv = suma(mult(s, G), mult(e, Y))        # (7, 11)
assert H(Rv[0], Rv[1], M) == e           # firma válida
```

# 5 Los dos ejemplos lado a lado

| | **Ejemplo 1:** $\mathbb{Z}_{23}^*$ | **Ejemplo 2:** $E(\mathbb{F}_{17})$ |
|---|---|---|
| Grupo y orden | $\langle 2\rangle \subset \mathbb{Z}_{23}^*$, $q = 11$ | $E : y^2 = x^3 + 2x + 2$, $q = 19$ |
| Generador | $g = 2$ | $G = (5, 1)$ |
| Clave privada | $x = 6$ | $x = 7$ |
| Clave pública | $y = g^{-6} = 9$ | $Y = -7G = (0, 11)$ |
| Aleatorio | $k = 7$ | $k = 10$ |
| Compromiso | $r = g^7 = 13$ | $R = 10G = (7, 11)$ |
| Reto | $e = H(13 \,\|\, 14) = 5$ | $e = H(7 \,\|\, 11 \,\|\, 14) = 16$ |
| Respuesta | $s = 7 + 6 \cdot 5 \equiv 4$ | $s = 10 + 7 \cdot 16 \equiv 8$ |
| Firma | $(4, 5)$ | $(8, 16)$ |
| Verificación | $g^4\, y^5 = 16 \cdot 8 \equiv 13$ | $8G + 16Y = (7, 11)$ |
| Reto recalculado | $5$ ✓ | $16$ ✓ |

### 5.1 Del juguete a la realidad

- **Tamaños.** En $\mathbb{Z}_p^*$ se usan $p$ de 2048 bits o más y $q$ de unos 256 bits. En curvas basta $q \approx 2^{256}$; Bitcoin usa secp256k1, con $p = 2^{256} - 2^{32} - 977$.
- **Hash.** Se usa una función criptográfica (SHA-256 con etiquetas, en BIP-340) e incluye la clave pública en el reto: $e = H(R \,\|\, Y \,\|\, M)$ [3].
- **Forma de la firma.** BIP-340 publica $(R, s)$ en lugar de $(s, e)$, con clave pública $X = xG$ y verificación $sG = R + eX$. Es la misma idea con el signo de la clave invertido; la forma $(R, s)$ es la que permite las multifirmas, porque los compromisos $R_i$ se pueden sumar.
- **El número $k$.** Debe ser impredecible y no repetirse jamás. Muchas implementaciones lo derivan de forma determinista a partir de la clave privada y el mensaje, lo que elimina la dependencia de un buen generador aleatorio.

## 6 Multifirma

### 6.1 La idea

En una multifirma, varios participantes autorizan juntos un mismo mensaje. El esquema que presentamos, *MuSig* [4], tiene una propiedad notable: el resultado es **una sola firma de Schnorr ordinaria**, verificable con **una sola clave pública conjunta**, exactamente con el procedimiento de la Sección 1. El verificador no puede saber cuántas personas firmaron.

Esto es posible porque la respuesta $s = k + x\,e$ es *lineal*: si dos personas usan el mismo reto $e$, la suma de sus respuestas es la respuesta de la suma de sus secretos,

$$(k_1 + x_1 e) + (k_2 + x_2 e) = (k_1 + k_2) + (x_1 + x_2)\, e.$$

En ECDSA la respuesta contiene $k^{-1}$, y esta suma simple no funciona.

**Definición: multifirma de Schnorr con dos firmantes (MuSig)**

Cada firmante $i \in \{1, 2\}$ tiene clave privada $x_i$ y pública $y_i = g^{-x_i}$.
**Paso 0: clave conjunta.** Con la lista de claves $L = (y_1, y_2)$ se calculan los coeficientes y la clave conjunta:

$$a_i = H(y_1 \,\|\, y_2 \,\|\, y_i), \qquad \tilde{y} = y_1^{a_1}\, y_2^{a_2}.$$

La clave privada conjunta sería $\tilde{x} = a_1 x_1 + a_2 x_2$, pero *nadie la conoce*: $\tilde{y} = g^{-\tilde{x}}$.
**Ronda 1: compromiso.** Cada firmante elige $k_i$ al azar, calcula $r_i = g^{k_i}$ y envía solo una huella $t_i = \text{SHA-256}(r_i)$.
**Ronda 2: revelación.** Recibidas todas las huellas, cada uno revela $r_i$; los demás comprueban la huella y todos calculan $r = r_1\, r_2$.
**Ronda 3: respuestas parciales.** Todos calculan el mismo reto $e = H(r \,\|\, M)$, y cada firmante envía

$$s_i = k_i + a_i\, x_i\, e \bmod q.$$

La firma es $(s, e)$ con $s = s_1 + s_2 \bmod q$.
**Verificación.** La misma de una firma individual, con la clave $\tilde{y}$: $r_v = g^s\, \tilde{y}^e$ y se acepta si $H(r_v \,\|\, M) = e$.

**Una simplificación consciente.** En el artículo de MuSig el reto incluye la clave conjunta, $e = H(\tilde{y} \,\|\, r \,\|\, M)$, y la prueba de seguridad depende de ello. Aquí usamos $e = H(r \,\|\, M)$ para que la verificación sea *literalmente* la de la Sección 1; la mecánica de las cuentas no cambia.

**Por qué funciona.**

$$g^s \, \tilde{y}^{\,e} = g^{(k_1+k_2)+(a_1x_1+a_2x_2)e} \cdot g^{-(a_1x_1+a_2x_2)e} = g^{k_1+k_2} = r_1 r_2 = r.$$

**Idea clave: para qué sirve cada pieza**

- **Los coeficientes** $a_i$ impiden que un participante elija su clave pública para *cancelar* la de los demás (ataque de clave maliciosa; véase el recuadro de la Sección 6.2).
- **La ronda de compromiso** impide que un participante elija su $r_i$ después de ver los de los demás. La primera versión de MuSig tenía solo dos rondas, y se demostró que su prueba de seguridad era incorrecta [27]; un atacante que abre muchas sesiones simultáneas puede forjar firmas (ataque de Wagner, hecho práctico con ROS [16]). MuSig2 elimina esta ronda de forma segura usando dos nonces por firmante [8].

En los ejemplos, el hash de juguete se usa para $a_i$ y para $e$. Para los compromisos de la Ronda 1 se muestran los primeros 8 dígitos hexadecimales de SHA-256: ese paso no forma parte de las cuentas, solo hay que comprobar que la huella coincide.

## 6.2 Ejemplo en $\mathbb{Z}_{23}^*$ con dos firmantes

Mismos parámetros del Ejemplo 1: $p = 23$, $q = 11$, $g = 2$ (Tabla 1). El mensaje es $M = 14$.

**Paso 0: claves individuales y clave conjunta**

| | **Alicia** | **Beto** |
|---|---|---|
| Clave privada $x_i$ | 6 | 4 |
| Clave pública $y_i = g^{-x_i}$ | $g^5 = 9$ | $g^7 = 13$ |
| Coeficiente $a_i = H(9 \,\|\, 13 \,\|\, y_i)$ | $9139 \bmod 11 = 9$ | $91313 \bmod 11 = 2$ |

(Divisiones: $9139 = 11 \cdot 830 + 9$ y $91313 = 11 \cdot 8301 + 2$.)
Clave conjunta $\tilde{y} = 9^9 \cdot 13^2 \bmod 23$:

$$9^2 = 81 \equiv 12, \quad 9^4 = 12^2 = 144 \equiv 6, \quad 9^8 = 6^2 = 36 \equiv 13, \quad 9^9 = 13 \cdot 9 = 117 \equiv 2,$$
$$13^2 = 169 \equiv 8, \quad \tilde{y} = 2 \cdot 8 = \boxed{16}.$$

Comprobación (que nadie puede hacer en la práctica): $\tilde{x} = 9 \cdot 6 + 2 \cdot 4 = 62 \equiv 7 \pmod{11}$ y $g^{-7} = g^4 = 16$.

### Rondas 1 y 2: compromisos y revelación

| | Alicia | Beto |
|---|---|---|
| Nonce secreto $k_i$ | 2 | 8 |
| $r_i = g^{k_i}$ | 4 | 3 |
| Ronda 1: envía $t_i = \text{SHA-256}(\texttt{com}\|r_i)$ | `51174631` | `21d0a206` |

En la Ronda 2 ambos revelan $r_i$, comprueban la huella del otro y calculan

$$r = r_1\, r_2 = 4 \cdot 3 = \boxed{12}.$$

### Ronda 3: reto y respuestas parciales

Reto común: $e = H(12 \,\|\, 14) = 1214 \bmod 11 = 4$ $(1214 = 11 \cdot 110 + 4)$.

$$\begin{aligned} s_1 &= k_1 + a_1 x_1 e = 2 + 9 \cdot 6 \cdot 4 = 218 \equiv 9 \pmod{11}, \\ s_2 &= k_2 + a_2 x_2 e = 8 + 2 \cdot 4 \cdot 4 = 40 \equiv 7 \pmod{11}, \\ s &= s_1 + s_2 = 16 \equiv 5 \pmod{11}. \end{aligned}$$

**Multifirma de $M = 14$:** $(s, e) = (5,\ 4)$ **con clave conjunta** $\tilde{y} = 16$

### Verificación (idéntica a la de una firma individual)

- $g^s = 2^5 = 32 \equiv 9$.
- $\tilde{y}^{\,e} = 16^4$: $16^2 = 256 \equiv 3$ y $16^4 = 3^2 = 9$.
- $r_v = 9 \cdot 9 = 81 \equiv 12$, y $e_v = H(12 \,\|\, 14) = 4 = e$ ✓.

El verificador solo vio la clave $\tilde{y} = 16$ y la firma $(5, 4)$: no hay rastro de que firmaron dos personas.

**Comprobar cada respuesta parcial.** Antes de sumar, cada firmante puede revisar la contribución del otro: debe cumplirse $g^{s_i}\, y_i^{\,a_i e} = r_i$. Para Alicia, $a_1 e = 36 \equiv 3$ y $2^9 \cdot 9^3 = 6 \cdot 16 = 96 \equiv 4 = r_1$ ✓. Para Beto, $a_2 e = 8$ y $2^7 \cdot 13^8 = 13 \cdot 2 = 26 \equiv 3 = r_2$ ✓(con $13^4 = 8^2 = 64 \equiv 18$ y $13^8 = 18^2 = 324 \equiv 2$). Así se detecta a un participante que envía una respuesta incorrecta.

### Advertencia: sin coeficientes: el ataque de clave maliciosa

Supongamos una versión ingenua con $\tilde{y} = y_1\, y_2$. Beto conoce de antemano $y_1 = 9$ y elige un secreto propio $x^* = 3$, con $y^* = g^{-3} = g^8 = 3$. En lugar de su clave honesta anuncia

$$y_2' = y^* \cdot y_1^{-1} = 3 \cdot 9^{-1} = 3 \cdot 18 = 54 \equiv 8 \pmod{23}$$

(pues $9 \cdot 18 = 162 \equiv 1$). La clave "conjunta" resulta $y_1\, y_2' = 9 \cdot 8 = 72 \equiv 3 = y^*$, y Beto puede firmar *solo* con $x^* = 3$, sin Alicia. Con los coeficientes $a_i = H(y_1 \,\|\, y_2 \,\|\, y_i)$ esto deja de funcionar: al cambiar $y_2$ cambian ambos exponentes, y Beto no puede prever cuál será $\tilde{y}$.

## 6.3 Ejemplo en la curva $y^2 = x^3 + 2x + 2$ sobre $\mathbb{F}_{17}$ con dos firmantes

Mismos parámetros del Ejemplo 2: $G = (5, 1)$, $q = 19$ (Tabla 3). En notación aditiva, la clave conjunta es $\tilde{Y} = a_1 Y_1 + a_2 Y_2$, el compromiso conjunto es $R = R_1 + R_2$ y la verificación es $R_v = sG + e\tilde{Y}$. Para que las cuentas sean cortas, el hash de los coeficientes usa solo las coordenadas $x$ de las claves: $a_i = H(x_{Y_1} \,\|\, x_{Y_2} \,\|\, x_{Y_i})$. El mensaje es $M = 14$.

### Paso 0: claves individuales y clave conjunta

| | **Alicia** | **Beto** |
|---|---|---|
| Clave privada $x_i$ | 7 | 8 |
| Clave pública $Y_i = -x_i G$ | $-(0,6) = (0,11)$ | $-(13,7) = (13,10)$ |
| Coeficiente $a_i = H(0 \,\|\, 13 \,\|\, x_{Y_i})$ | $130 \bmod 19 = 16$ | $1313 \bmod 19 = 2$ |

(Divisiones: $130 = 19 \cdot 6 + 16$ y $1313 = 19 \cdot 69 + 2$.)
$a_1 Y_1 = 16\,(0,11)$. Ya se calculó en el Paso 3b del Ejemplo 2, con cuatro doblados: $16Y_1 = (6,3)$.
$a_2 Y_2 = 2\,(13,10)$. Un doblado:

$$\lambda = \frac{3 \cdot 13^2 + 2}{2 \cdot 10} = \frac{509}{20} \equiv \frac{16}{3} = 16 \cdot 6 = 96 \equiv 11,$$

$$x_3 = 121 - 26 = 95 \equiv 10, \qquad y_3 = 11\,(13 - 10) - 10 = 23 \equiv 6.$$

Así, $2Y_2 = (10, 6)$.
$\tilde{Y} = (6,3) + (10,6)$.

$$\lambda = \frac{6-3}{10-6} = \frac{3}{4} = 3 \cdot 13 = 39 \equiv 5, \quad x_3 = 25 - 6 - 10 = 9, \quad y_3 = 5\,(6-9) - 3 = -18 \equiv 16.$$

Clave conjunta: $\boxed{\tilde{Y} = (9, 16)}$. Comprobación con la tabla: $\tilde{x} = 16 \cdot 7 + 2 \cdot 8 = 128 \equiv 14$, y $-14G = 5G = (9, 16)$.

### Rondas 1 y 2: compromisos y revelación

| | **Alicia** | **Beto** |
|---|---|---|
| Nonce secreto $k_i$ | 2 | 4 |
| $R_i = k_i G$ | $(6,3)$ | $(3,1)$ |
| Ronda 1: envía $t_i = \text{SHA-256}(\texttt{com}\,\|\,x_R, y_R)$ | `3095fa0f` | `2881cf99` |

En la Ronda 2 ambos revelan $R_i$ y calculan $R = R_1 + R_2 = (6,3) + (3,1)$:

$$\lambda = \frac{1-3}{3-6} = \frac{-2}{-3} = \frac{2}{3} = 2 \cdot 6 = 12,$$

$$x_3 = 144 - 9 = 135 \equiv 16, \qquad y_3 = 12\,(6 - 16) - 3 = -123 \equiv 13.$$

Compromiso conjunto: $\boxed{R = (16, 13)}$ (en la tabla, $6G$, pues $2 + 4 = 6$).

**Ronda 3: reto y respuestas parciales**

Reto común: $e = H(16 \,\|\, 13 \,\|\, 14) = 161314 \bmod 19 = 4$ $(161314 = 19 \cdot 8490 + 4)$.

$$\begin{aligned} s_1 &= k_1 + a_1 x_1 e = 2 + 16 \cdot 7 \cdot 4 = 450 \equiv 13 \pmod{19}, \\ s_2 &= k_2 + a_2 x_2 e = 4 + 2 \cdot 8 \cdot 4 = 68 \equiv 11 \pmod{19}, \\ s &= s_1 + s_2 = 24 \equiv 5 \pmod{19}. \end{aligned}$$

**Multifirma de $M = 14$:** $(s, e) = (5,\ 4)$ **con clave conjunta** $\tilde{Y} = (9, 16)$

**Verificación: $R_v = 5G + 4\tilde{Y}$**

$5G = 4G + G = (3, 1) + (5, 1)$, con $4G = (3, 1)$ del Paso 3a del Ejemplo 2:

$$\lambda = \frac{1-1}{5-3} = 0, \quad x_3 = 0 - 3 - 5 = -8 \equiv 9, \quad y_3 = 0 \cdot (3 - 9) - 1 \equiv 16.$$

Así, $5G = (9, 16)$.
$4\tilde{Y}$ **con dos doblados de** $(9, 16)$**.** Son los mismos doblados del Paso 3b del Ejemplo 2: $2\,(9, 16) = (7, 11)$ y $2\,(7, 11) = (5, 1)$. Así, $4\tilde{Y} = (5, 1)$.
$R_v = (9, 16) + (5, 1)$**:**

$$\lambda = \frac{1-16}{5-9} = \frac{-15}{-4} \equiv \frac{2}{13} = 2 \cdot 4 = 8,$$

$$x_3 = 64 - 9 - 5 = 50 \equiv 16, \qquad y_3 = 8\,(9 - 16) - 16 = -72 \equiv 13.$$

Así, $R_v = (16, 13) = R$ y $e_v = H(16 \,\|\, 13 \,\|\, 14) = 4 = e$ ✓.

**Lo que ve la cadena.** Una clave pública $(9, 16)$ y una firma $(5, 4)$ que se verifica como cualquier firma individual. Ni el número de firmantes ni sus claves aparecen en ningún lado; en Bitcoin, con Taproot, esto hace que un gasto de una multifirma 2-de-2 sea indistinguible de un pago ordinario y ocupe lo mismo.

## 6.4 Comprobación en Python

```
H = lambda q, *partes: int("".join(str(a) for a in partes)) % q

# ---------- Z_23^* ----------
p, q, g = 23, 11, 2
x1, x2 = 6, 4
y1, y2 = pow(g, -x1, p), pow(g, -x2, p)              # 9, 13
a1, a2 = H(q, y1, y2, y1), H(q, y1, y2, y2)          # 9, 2
yt = pow(y1, a1, p) * pow(y2, a2, p) % p             # 16
k1, k2 = 2, 8
r = pow(g, k1, p) * pow(g, k2, p) % p                # 12
e = H(q, r, 14)                                      # 4
s = ((k1 + a1*x1*e) + (k2 + a2*x2*e)) % q            # 5
assert H(q, pow(g, s, p) * pow(yt, e, p) % p, 14) == e

# ---------- Curva y^2 = x^3 + 2x + 2 sobre F_17 ----------
P, A, q, G = 17, 2, 19, (5, 1)
```

```
def suma(U, V):
    if U is None: return V
    if V is None: return U
    (x1_, y1_), (x2_, y2_) = U, V
    if x1_ == x2_ and (y1_ + y2_) % P == 0: return None
    if U == V: lam = (3*x1_*x1_ + A) * pow(2*y1_, -1, P) % P
    else:      lam = (y2_ - y1_) * pow(x2_ - x1_, -1, P) % P
    x3 = (lam*lam - x1_ - x2_) % P
    return (x3, (lam*(x1_ - x3) - y1_) % P)
def mult(n, U):
    R = None
    for b in bin(n)[2:]:
        R = suma(R, R)
        if b == "1": R = suma(R, U)
    return R
neg = lambda U: (U[0], -U[1] % P)

x1, x2 = 7, 8
Y1, Y2 = neg(mult(x1, G)), neg(mult(x2, G))           # (0,11), (13,10)
a1, a2 = H(q, Y1[0], Y2[0], Y1[0]), H(q, Y1[0], Y2[0], Y2[0])   # 16, 2
Yt = suma(mult(a1, Y1), mult(a2, Y2))                 # (9,16)
k1, k2 = 2, 4
R = suma(mult(k1, G), mult(k2, G))                    # (16,13)
e = H(q, R[0], R[1], 14)                              # 4
s = ((k1 + a1*x1*e) + (k2 + a2*x2*e)) % q             # 5
Rv = suma(mult(s, G), mult(e, Yt))                    # (16,13)
assert H(q, Rv[0], Rv[1], 14) == e
```

# 7 Ejercicios

1. Con los parámetros del Ejemplo 1 y la clave $x = 6$, firme el mensaje $M = 20$ usando $k = 5$. Verifique su firma.
2. Con los parámetros del Ejemplo 2 y la clave $x = 7$, firme el mensaje $M = 20$ usando $k = 4$. Verifique su firma usando únicamente $G$, $Y$ y el método de doblado y suma.
3. En el Ejemplo 1, alguien presenta la firma $(s, e) = (4, 6)$ para $M = 14$. ¿Es válida?
4. Complete a mano la Tabla 3 y compruebe que $19G = \mathcal{O}$. ¿Qué ocurre al calcular $18G + G$?
5. Explique por qué en el Ejemplo 2 el verificador puede calcular $R_v$ aunque no conozca $k$. ¿Qué parte de la verificación "reconstruye" el compromiso?

## Soluciones

1. $r = 2^5 = 9$; $e = 920 \bmod 11 = 7$; $s = 5 + 6 \cdot 7 = 47 \equiv 3$. Firma $(3, 7)$. Verificación: $2^3 = 8$, $9^7 = 9^4 \cdot 9^2 \cdot 9 = 6 \cdot 12 \cdot 9 = 648 \equiv 4$; $r_v = 8 \cdot 4 = 32 \equiv 9$ y $H(9 \,\|\, 20) = 7$ ✓.
2. $R = 4G = (3, 1)$; $e = 3120 \bmod 19 = 4$; $s = 4 + 7 \cdot 4 = 32 \equiv 13$. Firma $(13, 4)$. Verificación: $13G = (16, 4)$, $4Y = (7, 11)$ (Paso 3b), $(16, 4) + (7, 11) = (3, 1)$ y $H(3 \,\|\, 1 \,\|\, 20) = 4$ ✓.
3. $r_v = 2^4 \cdot 9^6 = 16 \cdot 3 = 48 \equiv 2$; $e_v = H(2 \,\|\, 14) = 214 \bmod 11 = 5 \neq 6$. **No** es válida.
4. $18G = (5, 16) = -G$, de modo que $18G + G = G + (-G) = \mathcal{O}$: los dos puntos tienen la misma $x$ y la recta que los une es vertical.

5. $sG + eY = (k + xe)G - exG = kG = R$: el término $eY$ cancela exactamente la parte $xe$ de la respuesta, y lo que queda es el compromiso.

# 8 Tarea

Los siguientes ejercicios usan parámetros *distintos* a los de los ejemplos, para que no puedan copiarse los resultados. Todos se resuelven a mano y se comprueban con el programa `tarea_schnorr.py` que aparece al final de esta sección. Use el mismo hash de juguete: concatenar los dígitos y reducir módulo $q$.

## Parte A: Schnorr en $\mathbb{Z}_{47}^*$

**Parámetros.** $p = 47$, $q = 23$, $g = 2$. Como $p - 1 = 46 = 2 \cdot 23$, el subgrupo generado por 2 tiene orden 23.

**Sugerencia**

Antes de empezar, construya la tabla de potencias $2^a \bmod 47$ para $a = 0, \ldots, 22$ (cada entrada es el doble de la anterior, reducido módulo 47) y compruebe que $2^{23} \equiv 1$. También le será útil la tabla de inversos módulo 23.

**A1. Firmar y verificar.** Con la clave privada $x = 9$:

(a) calcule la clave pública $y = g^{-x} \bmod 47$;

(b) firme el mensaje $M = 25$ con $k = 13$: obtenga $r$, $e$ y $s$;

(c) verifique su firma calculando $r_v = g^s y^e$ y $e_v$, usando únicamente $y$, $M$ y la firma.

**A2. ¿Cuáles son auténticas?** La clave pública de Beatriz es $y = 21$. Le llegan tres firmas del mensaje $M = 30$:

$$\text{A} = (s, e) = (6, 3), \qquad \text{B} = (7, 3), \qquad \text{C} = (4, 4).$$

Verifique cada una e indique cuáles son válidas. Para las que no lo sean, explique en qué paso falla la verificación.

**A3. El descuido del nonce.** La clave pública de Carlos es $y = 8$. Un observador nota que Carlos firmó dos mensajes con el mismo compromiso $r = 25$:

$$M_1 = 12 : \ (s_1, e_1) = (3, 5), \qquad M_2 = 40 : \ (s_2, e_2) = (11, 10).$$

(a) Compruebe que $e_1 = H(25 \,\|\, 12)$ y $e_2 = H(25 \,\|\, 40)$.

(b) Recupere la clave privada $x$ y el nonce $k$.

(c) Confirme que $g^{-x} \equiv 8$ y que $g^k \equiv 25 \pmod{47}$.

## Parte B: Schnorr en la curva $y^2 = x^3 + x + 6$ sobre $\mathbb{F}_{11}$

**Parámetros.** Campo $\mathbb{F}_{11}$, curva $E : \ y^2 = x^3 + x + 6$ (es decir, $a = 1$, $b = 6$), generador $G = (2, 7)$. La curva tiene 13 puntos contando $\mathcal{O}$; como 13 es primo, $q = 13$. Es un ejemplo clásico de libro de texto [28].

**Sugerencia**

Compruebe primero que $G$ está en la curva y construya la tabla de múltiplos $kG$ para $k = 1, \dots, 13$, verificando que $13G = \mathcal{O}$. Necesitará los inversos módulo 11 para las pendientes y los inversos módulo 13 para los escalares. Recuerde que en el hash se concatenan $x_R$, $y_R$ y $M$.

**B1. Firmar y verificar.** Con la clave privada $x = 5$:

(a) calcule la clave pública $Y = -xG$;

(b) firme el mensaje $M = 21$ con $k = 10$: obtenga $R$, $e$ y $s$;

(c) verifique su firma calculando $R_v = sG + eY$ *con doblado y suma a partir de* $Y$, sin usar que conoce $x$.

**B2. ¿Cuáles son auténticas?** La clave pública de Daniel es $Y = (10, 9)$. Le llegan tres firmas del mensaje $M = 9$:

$$\text{A} = (s, e) = (5, 7), \qquad \text{B} = (7, 7), \qquad \text{C} = (4, 5).$$

Verifique cada una e indique cuáles son válidas.

**B3. El descuido del nonce.** La clave pública de Elena es $Y = (8, 3)$. Firmó dos mensajes con el mismo compromiso $R = (3, 6)$:

$$M_1 = 5 : \ (s_1, e_1) = (2, 1), \qquad M_2 = 8 : \ (s_2, e_2) = (6, 4).$$

(a) Compruebe los dos retos con el hash de juguete.

(b) Recupere $x$ y $k$.

(c) Confirme que $-xG = (8, 3)$ y que $kG = (3, 6)$.

## Comprobación con Python

Copie el programa en un archivo `tarea_schnorr.py`, escriba sus resultados en el diccionario `RESPUESTAS` (los puntos como parejas, por ejemplo `(3, 5)`, y las firmas válidas como lista de letras, por ejemplo `["A"]`) y ejecútelo con `python3 tarea_schnorr.py`. Para cada valor, el programa responde *correcto* o *revisar*. Requiere Python 3.8 o posterior, por el uso de `pow(a, -1, m)` para calcular inversos modulares.

```
# Tarea: firma de Schnorr de juguete
# Llene RESPUESTAS con los valores que obtuvo a mano y ejecute:
#     python3 tarea_schnorr.py

RESPUESTAS = {
    # --- Z_47^*  (p=47, q=23, g=2) ---
    "Z1": {"y": None, "r": None, "e": None, "s": None, "rv": None},
    "Z2": {"validas": None},          # lista con las letras válidas, p. ej. ["A"]
    "Z3": {"x": None, "k": None},
    # --- E: y^2 = x^3 + x + 6 sobre F_11  (q=13, G=(2,7)) ---
    "E1": {"Y": None, "R": None, "e": None, "s": None, "Rv": None},
    "E2": {"validas": None},
    "E3": {"x": None, "k": None},
}

# ---------------------------------------------------------------------
# Herramientas (no es necesario modificar nada debajo de esta línea)
# ---------------------------------------------------------------------
```

```
def H(*partes, q):
    """Hash de juguete: concatena los dígitos y reduce módulo q."""
    return int("".join(str(a) for a in partes)) % q

# Z_p
p, qz, g = 47, 23, 2
def zp_verifica(y, M, s, e):
    rv = pow(g, s, p) * pow(y, e, p) % p
    return rv, H(rv, M, q=qz) == e

# Curva elíptica
P, A, qe, G = 11, 1, 13, (2, 7)
def suma(U, V):
    if U is None: return V
    if V is None: return U
    (x1, y1), (x2, y2) = U, V
    if x1 == x2 and (y1 + y2) % P == 0: return None
    if U == V: lam = (3*x1*x1 + A) * pow(2*y1, -1, P) % P
    else:      lam = (y2 - y1) * pow(x2 - x1, -1, P) % P
    x3 = (lam*lam - x1 - x2) % P
    return (x3, (lam*(x1 - x3) - y1) % P)
def mult(n, U):
    R = None
    for b in bin(n)[2:]:
        R = suma(R, R)
        if b == "1": R = suma(R, U)
    return R
neg = lambda U: (U[0], -U[1] % P)
def ec_verifica(Y, M, s, e):
    Rv = suma(mult(s, G), mult(e, Y))
    return Rv, Rv is not None and H(Rv[0], Rv[1], M, q=qe) == e

def revisa(nombre, obtenido, correcto):
    ok = obtenido == correcto
    print(f"  {nombre:8s}: {'correcto' if ok else 'revisar'}")
    return ok

# ---------------------------------------------------------------------
# Revisión
# ---------------------------------------------------------------------
print("Z1: x=9, k=13, M=25")
x, k, M = 9, 13, 25
y = pow(g, -x, p); r = pow(g, k, p); e = H(r, M, q=qz); s = (k + x*e) % qz
a = RESPUESTAS["Z1"]
revisa("y", a["y"], y); revisa("r", a["r"], r)
revisa("e", a["e"], e); revisa("s", a["s"], s)
revisa("r_v", a["rv"], zp_verifica(y, M, s, e)[0])

print("Z2: y=21, M=30")
firmas = {"A": (6, 3), "B": (7, 3), "C": (4, 4)}
validas = sorted(L for L, (s_, e_) in firmas.items() if zp_verifica(21, 30, s_, e_)[1])
revisa("válidas", sorted(RESPUESTAS["Z2"]["validas"] or []), validas)

print("Z3: y=8, firmas (3,5) para M=12 y (11,10) para M=40")
(s1, e1), (s2, e2) = (3, 5), (11, 10)
x = (s1 - s2) * pow(e1 - e2, -1, qz) % qz
k = (s1 - x*e1) % qz
revisa("x", RESPUESTAS["Z3"]["x"], x); revisa("k", RESPUESTAS["Z3"]["k"], k)
assert pow(g, -x, p) == 8
```

```
print("E1: x=5, k=10, M=21")
x, k, M = 5, 10, 21
Y = neg(mult(x, G)); R = mult(k, G)
e = H(R[0], R[1], M, q=qe); s = (k + x*e) % qe
a = RESPUESTAS["E1"]
revisa("Y", a["Y"], Y); revisa("R", a["R"], R)
revisa("e", a["e"], e); revisa("s", a["s"], s)
revisa("R_v", a["Rv"], ec_verifica(Y, M, s, e)[0])

print("E2: Y=(10,9), M=9")
firmas = {"A": (5, 7), "B": (7, 7), "C": (4, 5)}
validas = sorted(L for L, (s_, e_) in firmas.items() if ec_verifica((10, 9), 9, s_, e_)[1])
revisa("válidas", sorted(RESPUESTAS["E2"]["validas"] or []), validas)

print("E3: Y=(8,3), R=(3,6), firmas (2,1) para M=5 y (6,4) para M=8")
(s1, e1), (s2, e2) = (2, 1), (6, 4)
x = (s1 - s2) * pow(e1 - e2, -1, qe) % qe
k = (s1 - x*e1) % qe
revisa("x", RESPUESTAS["E3"]["x"], x); revisa("k", RESPUESTAS["E3"]["k"], k)
assert neg(mult(x, G)) == (8, 3) and mult(k, G) == (3, 6)
```

**Advertencia: sobre el entregable**

El programa solo confirma resultados: la tarea se evalúa con el procedimiento escrito a mano, incluyendo cada potencia, cada pendiente $\lambda$ y cada reducción modular.

# 9 Schnorr en la realidad: resumen, variantes y aplicaciones

Esta sección resume cómo se usa hoy la firma de Schnorr fuera del salón de clase, responde si la multifirma solo admite la forma "todos los $n$ firmantes" y reúne las variantes con aplicación práctica, sobre todo en criptomonedas y contratos inteligentes. Las referencias al final sustentan cada afirmación.

## 9.1 La firma individual en producción

- **Bitcoin (BIP-340).** Desde la activación de Taproot en noviembre de 2021, Bitcoin acepta firmas de Schnorr sobre la curva secp256k1 [3, 6, 7]. La firma ocupa 64 bytes ($x_R$ y $s$), la clave pública 32 bytes (solo la coordenada $x$) y el reto es un hash con etiqueta, $e = H_{\text{BIP0340/challenge}}(x_R \,\|\, x_P \,\|\, M)$. La verificación es $sG = R + eP$: la forma $(R, s)$ de la Sección 5.
- **Ed25519 / EdDSA.** Es una firma de Schnorr sobre una curva de Edwards, con nonce determinista $k = H(\text{secreto} \,\|\, M)$ [17, 18]. La usan Solana, Cardano (firmas de transacción), Algorand, Stellar, Near y Tezos, entre otras.
- **sr25519 (Polkadot, Kusama).** Schnorr sobre el grupo Ristretto de Curve25519.
- **Fuera de blockchain.** Ed25519 es estándar en SSH, TLS 1.3, firmas de paquetes de software y DNSSEC; Signal usa XEdDSA, una variante compatible.

**Idea clave: lo que el juguete omite**

Tamaños de 256 bits, hash con etiquetas que separan contextos, clave pública dentro del reto, nonce determinista o con aleatoriedad auxiliar, y codificaciones fijas que evitan ambigüedades.

Todo lo demás (compromiso, reto, respuesta $s = k + xe$) es exactamente lo que se hizo a mano.

## 9.2 ¿Solo existe la multifirma "$n$ de $n$"?

No. MuSig (Sección 6) produce una firma solo si *todos* los $n$ firmantes cooperan; si uno pierde su clave o se niega, los fondos quedan bloqueados. Para los demás casos hay tres familias de soluciones, que pueden combinarse.

**1. Firmas de umbral $t$ de $n$: FROST.** FROST [11, 12] reparte la clave conjunta con el esquema de Shamir: cada participante tiene una porción $x_i = f(i)$ de un polinomio $f$ de grado $t-1$ con $f(0) = \tilde{x}$. Cualquier subconjunto $S$ de $t$ firmantes produce una firma de Schnorr ordinaria, porque por interpolación de Lagrange

$$\tilde{x} = \sum_{i \in S} \lambda_i \, x_i, \qquad \lambda_i = \prod_{j \in S,\, j \neq i} \frac{j}{j - i},$$

y cada firmante aporta $s_i = k_i + \lambda_i x_i \, e$ (forma simplificada: en FROST cada firmante usa dos nonces combinados con un "factor de enlace" que depende del mensaje y de todo el grupo, igual que en MuSig2). Como en MuSig, nadie reconstruye $\tilde{x}$ y el verificador ve una sola firma.

- Se publicó como RFC 9591 (IRTF/CFRG) en junio de 2024.
- La generación distribuida de claves (DKG) evita que alguien conozca $\tilde{x}$ al crearla; para Bitcoin existe la propuesta ChillDKG [14].
- ROAST [13] envuelve a FROST para que la firma termine aun si algunos participantes no responden o actúan de mala fe.
- La Zcash Foundation mantiene una implementación en Rust con variantes para varias curvas, incluida la que usa Zcash para autorizar gastos [15].

**2. Políticas en el árbol de scripts de Taproot.** Una salida Taproot compromete una clave interna $P$ y un árbol de scripts (MAST) en una sola clave [6]:

$$Q = P + H_{\text{TapTweak}}(P \,\|\, \text{raíz})\, G.$$

Se puede gastar por la *ruta de clave* (una firma de Schnorr para $Q$, que puede ser una multifirma MuSig) o por la *ruta de script*, revelando solo la hoja usada. Con eso se arman políticas mixtas:

- Ruta de clave: MuSig 3 de 3 (caso normal, barato y privado).
- Hojas: cada par de firmantes como un MuSig 2 de 2, para obtener un 2 de 3 sin revelar a quien no firmó.
- Hoja con `OP_CHECKSIGADD` [7]: umbral $k$ de $n$ explícito, verificando cada firma de Schnorr por separado.
- Hoja de recuperación: una clave de respaldo que solo es válida después de cierto tiempo (*time-lock*). Miniscript permite componer estas políticas y analizarlas automáticamente.

**3. Variantes del propio MuSig.** MuSig2 [8, 9] reduce el protocolo a dos rondas y permite hacer la primera antes de conocer el mensaje; es el estándar en Bitcoin (BIP-327). MuSig-DN usa nonces deterministas con una prueba de conocimiento cero de que se generaron bien [10]. Las multifirmas *anidadas* permiten que un "firmante" sea a su vez un grupo.

| Solución | Política | En cadena | Uso típico |
|---|---|---|---|
| MuSig2 | $n$ de $n$ | Una firma, una clave | Canales de Lightning, cuentas compartidas |
| FROST (+ROAST) | $t$ de $n$ | Una firma, una clave | Custodia distribuida, federaciones, puentes |
| Taproot + hojas MuSig | $t$ de $n$ por combinaciones | Una firma + la hoja usada | Custodia colaborativa 2 de 3 |
| `OP_CHECKSIGADD` | $k$ de $n$ explícito | $k$ firmas visibles | Políticas auditables, muchos firmantes |
| Miniscript + *timelocks* | Umbral que cambia con el tiempo | La hoja usada | Herencia, recuperación |

## 9.3 Otras construcciones con aplicación real

La linealidad de $s = k + xe$ permite más que sumar firmas.

- **Firmas adaptadoras.** Una firma "incompleta" $s' = s - t$ que solo se vuelve válida al sumarle un secreto $t$, con $T = tG$ público. Publicar la firma completa revela $t = s - s'$. Esto permite:
  - *intercambios atómicos* entre cadenas sin contratos (*scriptless scripts*) [19];
  - *PTLC* en Lightning, que sustituyen los hashes de los HTLC y mejoran la privacidad de las rutas;
  - *Discreet Log Contracts* (DLC): contratos financieros (opciones, apuestas, coberturas) cuyo pago depende de la firma de un oráculo, sin que el oráculo sepa que el contrato existe [20].
- **Ajuste de claves (*tweaking*).** $Q = P + tG$ produce otra clave válida cuyo secreto es $x + t$. Taproot lo usa para esconder el árbol de scripts; las billeteras lo usan para derivar direcciones.
- **Verificación por lotes.** Verificar $m$ firmas juntas cuesta bastante menos que $m$ verificaciones separadas, combinando las ecuaciones con coeficientes aleatorios [3]. Acelera la validación de bloques.
- **Firmas ciegas.** El firmante firma sin ver el mensaje, la base del dinero electrónico (*ecash*) y de credenciales anónimas. Existen versiones ciegas de Schnorr, pero la variante ingenua es vulnerable al ataque ROS cuando hay sesiones concurrentes [16], por lo que los sistemas en producción suelen usar otras construcciones.
- **Firmas de anillo.** Monero usa CLSAG, una firma de anillo enlazable construida con la misma estructura compromiso–reto–respuesta [21].

## 9.4 Schnorr en criptomonedas y contratos inteligentes

**Bitcoin.** Bitcoin no tiene contratos inteligentes de propósito general, pero sí "contratos" expresados en scripts y en protocolos fuera de cadena, y en ambos Schnorr es central:

- **Taproot/Tapscript:** gastos por ruta de clave (una firma) o por hojas con `OP_CHECKSIG` y `OP_CHECKSIGADD` [6, 7].

- **Lightning:** los canales Taproot usan MuSig2 para el 2 de 2 del canal. En LND v0.21 (junio de 2026) pasaron de experimentales a listos para producción en canales privados; los canales públicos Taproot esperan aún el protocolo de difusión [22, 23].
- **DLC** e intercambios atómicos con firmas adaptadoras.
- **Custodia:** billeteras y custodios multifirma comienzan a adoptar MuSig2 y FROST para que el gasto conjunto parezca un pago ordinario y cueste lo mismo. Muchas federaciones aún usan multifirmas explícitas por script, que revelan la política en cadena.

**Ethereum y cadenas EVM.** La EVM no tiene un precompilado nativo de Schnorr; su firma de transacciones es ECDSA. Aun así se usa Schnorr dentro de contratos inteligentes:

- **Oráculos con firma agregada.** Chronicle (Scribe) verifica una firma de Schnorr agregada de muchos validadores aprovechando el precompilado `ecrecover` para calcular $sG - eP$ de forma barata [24]. Una sola verificación sustituye a muchas firmas ECDSA.
- **Abstracción de cuentas (ERC-4337).** Una cuenta inteligente define su propia regla de validación, de modo que puede exigir una firma de Schnorr (individual o MuSig) en lugar de ECDSA [25].

**Cardano.** Desde la bifurcación Valentine (2023), Plutus incluye la función nativa

`verifySchnorrSecp256k1Signature`,

compatible con BIP-340 [26]. Un contrato de Cardano puede verificar firmas de Schnorr producidas en el ecosistema de Bitcoin, lo que facilita puentes y aplicaciones entre cadenas. Las transacciones de Cardano, por su parte, se firman con Ed25519.

**Otras cadenas.** Solana, Algorand, Stellar, Near y Tezos firman con Ed25519; Polkadot con sr25519; Zcash usa RedDSA (una variante de Schnorr) para autorizar gastos y dispone de FROST para firmas de umbral. En todas ellas la firma de transacciones *es* una firma de Schnorr, aunque no siempre se le llame así.

## 9.5 Limitaciones y frontera

- **Interacción.** MuSig2 y FROST requieren rondas de comunicación y almacenar estado de nonces; reutilizar un nonce expone la clave, igual que en la Sección 2. Las bibliotecas serias (libsecp256k1-zkp, frost de la Zcash Foundation) existen para no implementarlo a mano.
- **Atribución.** Una multifirma agregada no revela quién firmó: es una ventaja de privacidad, pero dificulta auditar responsabilidades dentro del grupo.
- **Computación cuántica.** Schnorr, como ECDSA, se basa en el logaritmo discreto y no resiste el algoritmo de Shor. En Bitcoin se discute BIP-360, P2MR (*Pay-to-Merkle-Root*), un tipo de salida similar a Taproot pero sin ruta de clave, como paso hacia firmas poscuánticas; al momento de escribir este documento sigue en estado de borrador [29].

**Idea clave: en una línea**

Schnorr no es solo "otra firma": su linealidad es lo que hace posibles las multifirmas de una sola firma (MuSig2), los umbrales $t$ de $n$ (FROST), las políticas ocultas de Taproot y los contratos sin scripts (firmas adaptadoras, DLC). Por eso Bitcoin la adoptó con Taproot y otras cadenas la ofrecen como función nativa en sus contratos.

# 10 Firmas digitales especiales

Una firma digital ordinaria, como la de Schnorr, garantiza tres cosas: *autenticidad* (la produjo quien tiene la clave privada), *integridad* (el mensaje no cambió) y *no repudio* (el firmante no puede negarla). Muchos problemas reales piden algo más, o algo distinto: ocultar quién firmó, impedir que la firma se reenvíe a terceros, delegar la facultad de firmar, revelar solo parte de un documento firmado o sobrevivir a una computadora cuántica. Para cada uno de estos problemas existe una familia de **firmas especiales**. Esta sección presenta las principales, agrupadas por el problema que resuelven, con su origen, su idea central y dónde se usan hoy.

## 10.1 Privacidad del firmante

**Firmas ciegas** — Chaum, 1982 [30]

**Idea.** El firmante firma un mensaje que no puede ver; el solicitante "desciega" después la firma y obtiene una firma válida sobre su mensaje original. En la versión RSA, con clave pública $(n, e)$ y privada $d$:

$$m' = m\,r^e \bmod n, \qquad s' = (m')^d = m^d\,r, \qquad s = s'\,r^{-1} = m^d \bmod n.$$

El firmante vio $m'$, que no revela nada de $m$ porque $r$ es aleatorio.

**Problema que resuelve.** Que una autoridad avale algo (un pago, un voto, un "soy humano") sin poder rastrear después a quién se lo avaló.

**Uso real.** Dinero electrónico (*ecash*); tokens anónimos de Privacy Pass y de servicios como iCloud Private Relay, estandarizados para RSA en el RFC 9474 [31]; protocolos de voto electrónico. Como se vio en la Sección 9, la versión ciega ingenua de Schnorr es vulnerable al ataque ROS.

**Firmas de anillo** — Rivest, Shamir y Tauman, 2001 [32]

**Idea.** El firmante elige un conjunto ("anillo") de claves públicas que incluye la suya y produce una firma que prueba que *alguna* de esas claves firmó, sin decir cuál. No hay administrador ni preparación previa: los demás miembros ni siquiera se enteran.

**Problema que resuelve.** Anonimato espontáneo dentro de un grupo; el ejemplo clásico es el denunciante que prueba ser miembro de un gabinete sin identificarse.

**Uso real.** Las firmas de anillo *enlazables* añaden una "imagen de clave" que delata si la misma clave firma dos veces. Monero las usa (CLSAG) para ocultar al remitente y, a la vez, impedir el doble gasto [21].

**Firmas de grupo y atestación anónima** — Chaum y van Heyst, 1991 [33]

**Idea.** Un administrador registra a los miembros; cualquiera de ellos firma en nombre del grupo de forma anónima, pero el administrador (o una autoridad de apertura) puede revelar quién firmó si hay un abuso.

**Problema que resuelve.** Anonimato con rendición de cuentas: privacidad para el uso normal y trazabilidad ante una disputa.

**Uso real.** La *atestación anónima directa* (DAA) [34] es una variante que permite a un chip probar que es un componente legítimo sin revelar cuál: está en los TPM 2.0 (ECDAA) y fue la base de Intel EPID para la atestación remota de SGX. Las firmas BBS [35] nacieron como firmas de grupo cortas.

## 10.2 Control sobre quién puede verificar

**Firmas no repudiables (*undeniable*)** Chaum y van Antwerpen, 1989 [36]

**Idea.** La firma no se puede verificar a solas: hace falta un protocolo interactivo con el firmante, quien puede confirmarla o, si es falsa, probar que no es suya (protocolo de desconocimiento).

**Problema que resuelve.** Que un documento firmado (por ejemplo, un software con licencia o una oferta confidencial) no pueda mostrarse a terceros como prueba sin la cooperación del firmante.

**Uso real.** Principalmente académico y en esquemas de licencias de software; inspiró las firmas de verificador designado.

**Firmas de verificador designado** Jakobsson, Sako e Impagliazzo, 1996 [37]

**Idea.** Solo un verificador elegido queda convencido, porque él mismo podría haber fabricado una firma indistinguible; por eso no puede usarla para convencer a nadie más.

**Problema que resuelve.** Autenticación sin dejar una prueba transferible: útil cuando el firmante no quiere que su mensaje sea una evidencia pública.

**Uso real.** Voto electrónico resistente a la coerción (el votante no puede probar a un tercero por quién votó) y protocolos de mensajería con negación plausible, como la autenticación de Signal y OTR, que persiguen el mismo objetivo con otras herramientas.

## 10.3 Delegación y firma colectiva

**Firmas por apoderado (*proxy*)** Mambo, Usuda y Okamoto, 1996 [38]

**Idea.** El titular entrega a un apoderado una "orden de delegación" firmada; el apoderado firma en su nombre, y el verificador comprueba a la vez la firma, la delegación y sus límites (tipo de documento, vigencia).

**Problema que resuelve.** Delegar la facultad de firmar sin entregar la clave privada, como un poder notarial digital.

**Uso real.** Sistemas de flujo de trabajo y agentes móviles; en blockchain, la misma idea aparece como *session keys* en cuentas inteligentes (ERC-4337), que autorizan a una clave temporal con permisos acotados.

**Multifirmas y firmas de umbral** Secciones 6 y 9

**Idea.** Varios firmantes producen una sola firma ($n$ de $n$ con MuSig2, $t$ de $n$ con FROST) verificable con una sola clave conjunta.

**Problema que resuelve.** Eliminar el punto único de falla en la custodia de claves.

**Uso real.** Bitcoin (Taproot, Lightning), custodios institucionales, federaciones y puentes entre cadenas.

**Firmas agregadas (BLS)** Boneh, Lynn y Shacham, 2001; Boneh *et al.*, 2003 [39, 40]

**Idea.** Con un emparejamiento bilineal $e$, la firma BLS de $m$ es $\sigma = x\,H(m)$ y se verifica con $e(\sigma, g_2) = e(H(m), pk)$. Firmas de *distintas* personas sobre *distintos* mensajes se suman en una sola de 48 o 96 bytes, sin interacción entre ellas.

**Problema que resuelve.** Comprimir miles de firmas en una para ahorrar espacio y verificación.

**Uso real.** El consenso de Ethereum agrega con BLS las firmas de cientos de miles de validadores; también Chia, Filecoin, ICP y la aleatoriedad pública de drand. Requiere defensas contra el ataque de clave maliciosa (pruebas de posesión), el mismo problema de la Sección 6.2.

## 10.4 Credenciales y divulgación selectiva

**Firmas CL y BBS** Camenisch y Lysyanskaya, 2002; Boneh, Boyen y Shacham, 2004 [35, 41]

**Idea.** Una sola firma sobre una *lista* de atributos (nombre, fecha de nacimiento, nacionalidad, …). El titular puede después *demostrar en conocimiento cero* que posee una firma válida revelando solo algunos atributos, o solo un predicado ("soy mayor de 18"), y cada presentación es distinta, de modo que no puede ser rastreada.

**Problema que resuelve.** Credenciales digitales que respetan la privacidad: mostrar lo mínimo necesario y evitar que el emisor o los verificadores sigan al usuario.

**Uso real.** Las firmas CL son la base de Idemix (IBM) y de AnonCreds (Hyperledger), usadas en identidad digital autosoberana. BBS se está estandarizando en el IRTF (borrador 12, septiembre de 2026) para credenciales verificables del W3C [42].

**Firmas basadas en atributos** Maji, Prabhakaran y Rosulek, 2011 [43]

**Idea.** La firma prueba que el firmante posee atributos que satisfacen una política ("médico $y$ del hospital X") sin revelar su identidad ni cuáles atributos exactos tiene.

**Problema que resuelve.** Autorización por rol o por propiedades, sin identificación personal.

**Uso real.** Control de acceso en salud y gobierno, y mensajería anónima pero calificada; en su mayoría prototipos y pilotos.

**Firmas redactables y sanitizables** Johnson *et al.*, 2002; Ateniese *et al.*, 2005 [44, 45]

**Idea.** Partes del documento firmado pueden *tacharse* (redactables) o *modificarse* por un tercero autorizado (sanitizables) sin invalidar la firma del autor sobre el resto.

**Problema que resuelve.** Publicar documentos oficiales con datos personales tachados, o expedientes médicos anonimizados, conservando la garantía de autenticidad.

**Uso real.** Transparencia gubernamental, expedientes clínicos y cadenas de suministro; investigación aplicada con pilotos.

## 10.5 Gestión de claves y seguridad a largo plazo

**Firmas basadas en identidad** — Shamir, 1984 [46]

**Idea.** La clave pública *es* la identidad (un correo, un número de empleado); una autoridad de confianza genera la clave privada correspondiente.

**Problema que resuelve.** Eliminar los certificados y la infraestructura de clave pública (PKI): para verificar basta conocer la identidad.

**Uso real.** Sistemas cerrados (redes corporativas, internet de las cosas, comunicaciones militares). Su limitación es que la autoridad conoce todas las claves privadas (*key escrow*).

**Firmas con seguridad hacia adelante** — Bellare y Miner, 1999 [47]

**Idea.** La clave pública es fija, pero la privada evoluciona en cada periodo y la anterior se borra; robar la clave de hoy no permite falsificar firmas fechadas en periodos pasados.

**Problema que resuelve.** Limitar el daño de un robo de clave.

**Uso real.** Bitácoras seguras (*secure logging*) y consenso: las claves de los productores de bloques de Cardano (KES) y de Algorand evolucionan con el tiempo por este motivo.

**Firmas basadas en hash** — Lamport, 1979; Merkle, 1989 [48, 49]

**Idea.** Solo requieren una función hash. La firma de Lamport revela, para cada bit del mensaje, uno de dos secretos previamente comprometidos; sirve una sola vez. Un árbol de Merkle agrupa muchas claves de un solo uso bajo una sola clave pública.

**Problema que resuelve.** Seguridad que no depende de problemas algebraicos: resisten a una computadora cuántica.

**Uso real.** XMSS y LMS (RFC 8391 y 8554, con estado) se recomiendan para firmar *firmware* [50, 51]; SLH-DSA (SPHINCS+, sin estado) es el estándar FIPS 205 [52]. En blockchain las usan QRL (XMSS) y la versión original de IOTA (Winternitz). Su costo: firmas de varios kilobytes.

**Firmas basadas en retículas** — NIST, 2024 [53]

**Idea.** Su seguridad se basa en problemas de retículas (como encontrar vectores cortos), que se consideran difíciles incluso para una computadora cuántica. ML-DSA (Dilithium) usa el paradigma "Fiat–Shamir con abortos", un pariente cercano de Schnorr: compromiso, reto y respuesta, repitiendo si la respuesta revelaría información del secreto.

**Problema que resuelve.** Sustituir a RSA, ECDSA y Schnorr antes de que existan computadoras cuánticas capaces de romperlos.

**Uso real.** ML-DSA es el estándar FIPS 204 (agosto de 2024) y ya se integra en navegadores, sistemas operativos y módulos criptográficos. FN-DSA (Falcon), con firmas más compactas, sigue como borrador FIPS 206 [54]. En blockchain, su tamaño (unos 2.4 KB para ML-DSA frente a 64 bytes de Schnorr) es el principal obstáculo para adoptarlas.

## 10.6 Resumen

| Tipo de firma | Problema que resuelve | Uso real o relevante |
|---|---|---|
| Ciega | Avalar sin poder rastrear | Ecash, Privacy Pass (RFC 9474), voto |
| De anillo (enlazable) | Anonimato espontáneo sin doble gasto | Monero (CLSAG) |
| De grupo / DAA | Anonimato con posibilidad de apertura | TPM 2.0, Intel EPID |
| No repudiable | La firma no se transfiere sin el firmante | Licencias, investigación |
| Verificador designado | Autenticación sin prueba pública | Voto anticoerción |
| Por apoderado | Delegar sin entregar la clave | Flujos de trabajo, *session keys* |
| Multifirma / umbral | Sin punto único de falla | Bitcoin, custodia, puentes |
| Agregada (BLS) | Comprimir miles de firmas | Consenso de Ethereum, drand |
| CL / BBS | Divulgación selectiva y no rastreable | AnonCreds, Idemix, credenciales W3C |
| Basada en atributos | Autorizar por propiedades, no por identidad | Control de acceso |
| Redactable / sanitizable | Tachar o editar sin perder autenticidad | Documentos públicos, salud |
| Basada en identidad | Prescindir de certificados | Sistemas cerrados, IoT |
| Hacia adelante | Proteger el pasado ante un robo de clave | Bitácoras, Cardano KES |
| Basada en hash | Seguridad poscuántica conservadora | Firmware, SLH-DSA, QRL |
| Basada en retículas | Sustituto poscuántico general | ML-DSA (FIPS 204) |

**Idea clave: el hilo común**

Casi todas estas firmas se construyen con las mismas piezas que este documento calculó a mano: un compromiso, un reto obtenido con un hash y una respuesta lineal en el secreto. Cambiar qué se compromete, quién conoce el secreto o qué se prueba en conocimiento cero es lo que convierte una firma de Schnorr en una firma de anillo, de grupo, ciega o de credenciales.

# Referencias


[1] C. P. Schnorr. Efficient Signature Generation by Smart Cards. *Journal of Cryptology*, 4(3):161–174, 1991.

[2] C. Paar y J. Pelzl. *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer, 2010. (La curva del Ejemplo 2 aparece en el capítulo 9.)

[3] P. Wuille, J. Nick y T. Ruffing. BIP-340: Schnorr Signatures for secp256k1. *Bitcoin Improvement Proposals*, 2020.

[4] G. Maxwell, A. Poelstra, Y. Seurin y P. Wuille. Simple Schnorr Multi-Signatures with Applications to Bitcoin. *Designs, Codes and Cryptography*, 87(9):2139–2164, 2019.

[5] D. Hankerson, A. Menezes y S. Vanstone. *Guide to Elliptic Curve Cryptography*. Springer, 2004.

[6] P. Wuille, J. Nick y A. Towns. BIP-341: Taproot: SegWit version 1 spending rules. *Bitcoin Improvement Proposals*, 2020.

[7] P. Wuille, J. Nick y A. Towns. BIP-342: Validation of Taproot Scripts. *Bitcoin Improvement Proposals*, 2020.

[8] J. Nick, T. Ruffing y Y. Seurin. MuSig2: Simple Two-Round Schnorr Multi-Signatures. En *CRYPTO 2021*, LNCS 12825, Springer, 2021.

[9] J. Nick, T. Ruffing y E. Jin. BIP-327: MuSig2 for BIP340-compatible Multi-Signatures. *Bitcoin Improvement Proposals*, 2023.

[10] J. Nick, T. Ruffing, Y. Seurin y P. Wuille. MuSig-DN: Schnorr Multi-Signatures with Verifiably Deterministic Nonces. En *ACM CCS 2020*, 2020.

[11] C. Komlo e I. Goldberg. FROST: Flexible Round-Optimized Schnorr Threshold Signatures. En *SAC 2020*, LNCS 12804, Springer, 2020.

[12] D. Connolly, C. Komlo, I. Goldberg y C. A. Wood. The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures. RFC 9591, IRTF, junio de 2024.

[13] T. Ruffing, V. Ronge, E. Jin, J. Schneider-Bensch y D. Schröder. ROAST: Robust Asynchronous Schnorr Threshold Signatures. En *ACM CCS 2022*, 2022.

[14] J. Nick, T. Ruffing y otros. ChillDKG: Distributed Key Generation for FROST. Borrador de BIP, lista *bitcoindev*, 2024.

[15] Zcash Foundation. *frost*: implementación en Rust de FROST (RFC 9591). `https://github.com/ZcashFoundation/frost`.

[16] F. Benhamouda, T. Lepoint, J. Loss, M. Orrù y M. Raykova. On the (in)security of ROS. En *EUROCRYPT 2021*, Springer, 2021.

[17] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe y B.-Y. Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.

[18] S. Josefsson e I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, IRTF, 2017.

[19] A. Poelstra. Mimblewimble and Scriptless Scripts. Presentación, *MIT Bitcoin Expo*, marzo de 2017.

[20] T. Dryja. Discreet Log Contracts. Documento técnico, MIT Digital Currency Initiative, 2017.

[21] B. Goodell, S. Noether y A. Blue. Concise Linkable Ring Signatures and Forgery Against Adversarial Keys. *Cryptology ePrint Archive*, 2019/654.

[22] Lightning Labs. Announcing LND v0.21-beta. 11 de junio de 2026. `https://lightning.engineering/posts/2026-06-11-lnd-0.21-launch/`.

[23] Bitcoin Optech. Simple taproot channels. `https://bitcoinops.org/en/topics/simple-taproot-channels/`.

[24] Chronicle Protocol. Scribe: Schnorr signature scheme. `https://github.com/chronicleprotocol/scribe/blob/main/docs/Schnorr.md`.

[25] V. Buterin *et al.* ERC-4337: Account Abstraction Using Alt Mempool. *Ethereum Improvement Proposals*, 2021.

[26] K. Ross, M. Peyton-Jones e I. Querejeta Azurmendi. CIP-49: ECDSA and Schnorr signatures in Plutus Core. *Cardano Improvement Proposals*. `https://cips.cardano.org/cip/CIP-49`.

[27] M. Drijvers, K. Edalatnejad, B. Ford, E. Kiltz, J. Loss, G. Neven e I. Stepanovs. On the Security of Two-Round Multi-Signatures. En *IEEE Symposium on Security and Privacy*, 2019.

[28] D. R. Stinson y M. B. Paterson. *Cryptography: Theory and Practice*, 4.ª ed. CRC Press, 2018.

[29] H. Beast, E. Heilman e I. Foxen Duke. BIP-360: Pay-to-Merkle-Root (P2MR). *Bitcoin Improvement Proposals*, borrador. `https://bips.dev/360/`.

[30] D. Chaum. Blind Signatures for Untraceable Payments. En *Advances in Cryptology: Proceedings of CRYPTO '82*, Plenum Press, 1983, pp. 199–203.

[31] F. Denis, F. Jacobs y C. A. Wood. RSA Blind Signatures. RFC 9474, IRTF, octubre de 2023.

[32] R. L. Rivest, A. Shamir e Y. Tauman. How to Leak a Secret. En *ASIACRYPT 2001*, LNCS 2248, Springer, 2001.

[33] D. Chaum y E. van Heyst. Group Signatures. En *EUROCRYPT '91*, LNCS 547, Springer, 1991.

[34] E. Brickell, J. Camenisch y L. Chen. Direct Anonymous Attestation. En *ACM CCS 2004*, 2004.

[35] D. Boneh, X. Boyen y H. Shacham. Short Group Signatures. En *CRYPTO 2004*, LNCS 3152, Springer, 2004.

[36] D. Chaum y H. van Antwerpen. Undeniable Signatures. En *CRYPTO '89*, LNCS 435, Springer, 1990.

[37] M. Jakobsson, K. Sako y R. Impagliazzo. Designated Verifier Proofs and Their Applications. En *EUROCRYPT '96*, LNCS 1070, Springer, 1996.

[38] M. Mambo, K. Usuda y E. Okamoto. Proxy Signatures for Delegating Signing Operation. En *ACM CCS '96*, 1996.

[39] D. Boneh, B. Lynn y H. Shacham. Short Signatures from the Weil Pairing. En *ASIACRYPT 2001*, LNCS 2248, Springer, 2001.

[40] D. Boneh, C. Gentry, B. Lynn y H. Shacham. Aggregate and Verifiably Encrypted Signatures from Bilinear Maps. En *EUROCRYPT 2003*, LNCS 2656, Springer, 2003.

[41] J. Camenisch y A. Lysyanskaya. A Signature Scheme with Efficient Protocols. En *SCN 2002*, LNCS 2576, Springer, 2003.

[42] T. Looker, V. Kalos, A. Whitehead y M. Lodder. The BBS Signature Scheme. Internet-Draft `draft-irtf-cfrg-bbs-signatures-12`, IRTF, septiembre de 2026.

[43] H. K. Maji, M. Prabhakaran y M. Rosulek. Attribute-Based Signatures. En *CT-RSA 2011*, LNCS 6558, Springer, 2011.

[44] R. Johnson, D. Molnar, D. Song y D. Wagner. Homomorphic Signature Schemes. En *CT-RSA 2002*, LNCS 2271, Springer, 2002.

[45] G. Ateniese, D. H. Chou, B. de Medeiros y G. Tsudik. Sanitizable Signatures. En *ESORICS 2005*, LNCS 3679, Springer, 2005.

[46] A. Shamir. Identity-Based Cryptosystems and Signature Schemes. En *CRYPTO '84*, LNCS 196, Springer, 1985.

[47] M. Bellare y S. K. Miner. A Forward-Secure Digital Signature Scheme. En *CRYPTO '99*, LNCS 1666, Springer, 1999.

[48] L. Lamport. Constructing Digital Signatures from a One-Way Function. Informe técnico SRI-CSL-98, SRI International, 1979.

[49] R. C. Merkle. A Certified Digital Signature. En *CRYPTO '89*, LNCS 435, Springer, 1990.

[50] A. Hülsing, D. Butin, S. Gazdag, J. Rijneveld y A. Mohaisen. XMSS: eXtended Merkle Signature Scheme. RFC 8391, IRTF, 2018.

[51] D. McGrew, M. Curcio y S. Fluhrer. Leighton-Micali Hash-Based Signatures. RFC 8554, IRTF, 2019.

[52] NIST. *FIPS 205: Stateless Hash-Based Digital Signature Standard*. Agosto de 2024.

[53] NIST. *FIPS 204: Module-Lattice-Based Digital Signature Standard*. Agosto de 2024.

[54] NIST. *FIPS 206 (borrador): FN-DSA, FFT over NTRU-Lattice-Based Digital Signature Algorithm*. En preparación.